

Analyse syntaxique

Cyril Rabat

`cyril.rabat@univ-reims.fr`

Licence 3 Informatique - Info0602 - Langages et compilation

2022-2023



Cours n°3

Qu'est-ce qu'un analyseur syntaxique ?
Grammaires

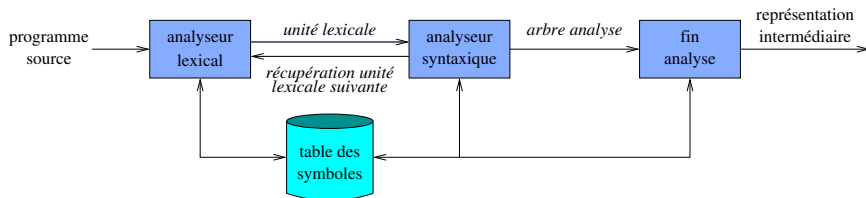
Version 26 février 2023

Table des matières

- 3 L'analyse syntaxique
 - Un analyseur syntaxique
 - Les grammaires
 - Analyse syntaxique descendante
 - Analyseurs prédictifs

Un analyseur syntaxique

- Récupération des unités lexicales depuis l'analyseur lexical
- Vérification de la structuration
 - ↪ Utilisation d'une **grammaire**, constituée de **règles de production**
- Gestion des erreurs :
 - ↪ Signalement + support des erreurs courantes



Une règle de production

- Exemple de la conditionnelle en C :

if (*expression*) *instruction* **else** *instruction*

↪ Concaténation du mot-clé **if**, d'une parenthèse ouvrante, d'une expression, d'une parenthèse fermante, *etc.*

- Ce qui peut s'exprimer sous la forme d'une règle :

inst → **if** (*expr*) *inst* **else** *inst*

↪ C'est une **règle de production**

- if, (,) sont des unités lexicales appelées **symboles terminaux**
 - ↪ Ces symboles apparaissent dans le langage
- expr*, *inst* sont des suites d'unités lexicales appelées **symboles non terminaux**
 - ↪ Ces symboles sont utilisés pour construire la grammaire

Définition d'un langage à l'aide de règles

- Exemple : soit le langage $L = \{a^n b^n \mid n \geq 0\}$
 - $\hookrightarrow \epsilon \in L$
 - $\hookrightarrow \{a\}L\{b\} \subseteq L$
- Nous pouvons le décrire sous forme de règles :

$$\begin{array}{lcl} L & \rightarrow & \epsilon \\ L & \rightarrow & aLb \end{array}$$
 - $\hookrightarrow$ Que l'on peut écrire sous forme condensée : $L \rightarrow \epsilon \mid aLb$
- Interprétation :
 - $\hookrightarrow L \rightarrow \epsilon \Leftrightarrow \epsilon \in L$
 - $\hookrightarrow L \rightarrow aLb \Leftrightarrow \forall w \in L, awb \in L$ avec a et b des symboles terminaux
- Ces règles forment une **grammaire** et permettent d'engendrer les mots d'un langage

Une grammaire

Définition : grammaire

Une **grammaire** est définie par le quadruplet $G = (T, N, R, S)$ où :

- T est un ensemble de symboles terminaux
- N est un ensemble de symboles non terminaux
- $R \subseteq ((T \cup N)^+ \times (T \cup N)^*)$ est un ensemble de règles de réécriture ou de production
- $S \in N$ est le symbole de départ ou **axiome**

- Les symboles de N n'apparaissent pas dans les mots générés
- Les règles sont de la forme

$$u1 \rightarrow u2 \text{ avec } u1 \in (N \cup T)^+ \text{ et } u2 \in (N \cup T)^*$$

$\hookrightarrow$ Si $u2 \in T^*$, $u1 \rightarrow u2$ est une **règle terminale**

- C'est à partir de S que la génération de mots commence

Remarques

- Langage défini par une grammaire = ensemble des mots obtenus (dérivés) à partir de l'axiome par application des règles de la grammaire
- Règles de notation :
 - Les symboles terminaux sont représentés par des minuscules
 - Les symboles non-terminaux par des majuscules

Mot reconnu par une grammaire

- Soit G la grammaire suivante :

$$G = (T = \{a, b\}, N = \{S\}, R = \{S \rightarrow \epsilon, S \rightarrow aSb\}, S)$$

- G définit le langage $L = \{a^n b^n \mid n \geq 0\}$
- Le mot $aabb$ fait partie du langage L car :

- ① $\underline{S}$
- ② $a\underline{S}b \ (S \rightarrow aSb)$
- ③ $aa\underline{S}bb \ (S \rightarrow aSb)$
- ④ $aabb \ (S \rightarrow \epsilon)$

Dérivation en une étape

Définition : dérivation (en une étape)

Soit une grammaire $G = (T, N, R, S)$ et $u \in (T \cup N)^+$, $v \in (T \cup N)^$. G permet de dériver u en v en une étape (noté $u \Rightarrow v$) si et seulement si :*

- *$u = xu'y$ (u peut être décomposé en x , u' , y ; x et y éventuellement vides)*
- *$v = xv'y$ (v peut être décomposé en x , v' , y éventuellement vides)*
- *$u' \rightarrow v' \in R$*

Dérivation et langage généré

Définition : dérivation

$G = (T, N, R, S)$ permet de dériver u en v (noté $u \Rightarrow^* v$) si et seulement si $\exists k \geq 0$ et $v_0, \dots, v_{k-1} \in (T \cup N)^+$ et $v_k \in (T \cup N)^*$ tel que :

- $u = v_0$
- $v = v_k$
- $v_i \Rightarrow v_{i+1}$ pour $0 \leq i < k$

Définition : langage généré

Le langage $L(G)$ généré par une grammaire $G = (T, N, R, S)$ est l'ensemble des mots qui peuvent être générés par G :

$$L(G) = \{v \in T^* \mid S \Rightarrow^* v\}$$

Classification des grammaires (1/2)

- Classification en fonction de la forme des règles
 - ↪ Définie en 1957 par Noam Chomsky
- **Type 0** : pas de restriction
 - ↪ Règles de la forme $w \rightarrow v$
- **Type 1** : grammaires sensibles au contexte (dites contextuelles)
 - ↪ Règles de la forme $uAv \rightarrow uwv$
- **Type 2** : grammaires hors-contexte
 - ↪ Règles de la forme $A \rightarrow u$
- **Type 3** : grammaires régulières
 - À droite
 - ↪ Règles de la forme $A \rightarrow aB$ ou $A \rightarrow a$
 - À gauche
 - ↪ Règles de la forme $A \rightarrow Ba$ ou $A \rightarrow a$

Avec $A, B \in N$, $u, v \in (N \cup T)^*$, $w \in (N \cup T)^+$, $a \in T^*$

Classification des grammaires (2/2)

- Les grammaires de type 3 génèrent les langages réguliers
- Les grammaires de type 2 génèrent les langages hors-contexte
- Les grammaires de type 1 génèrent les langages contextuels
- Les grammaires de type 0 permettent de générer tout langage "**décidables**"
↪ Langages reconnus en temps fini par une machine
- Les langages qui ne peuvent être générés par une grammaire de type 0 sont dits "**indécidables**"
- L'ensemble des langages générés par des grammaires de type n est strictement inclus dans celui des grammaires de type $n - 1$ (avec $n \in \{1, 2, 3\}$)

Grammaires et reconnaissance par des automates

- Chaque type de grammaire est reconnu par un type spécifique d'automate qui reconnaît les langages générés
 - Langages réguliers : automates finis
 - Langages hors-contexte : automates finis à pile
 - Autres langages : machines de Turing

Grammaires hors-contexte et régulière

Définition : grammaire hors-contexte

Une grammaire est hors-contexte si le membre de gauche de toute règle est un non terminal :

$$A \rightarrow w, \text{ avec } A \in N \text{ et } w \in (N \cup T)^*$$

Définition : grammaire régulière droite

Une grammaire est régulière droite (ou linéaire droite) si toutes ses productions vérifient une des deux formes :

$$A \rightarrow aB \text{ ou } A \rightarrow a \text{ avec } A, B \in N \text{ et } a \in T^*$$

Grammaires régulières droite et gauche (1/2)

- La majorité des langages de programmation peuvent être décrits par un langage régulier
- Lors de la lecture des symboles d'un mot à analyser de la gauche vers la droite :
 - Grammaire régulière droite : analyse descendante
↔ De l'axiome vers le mot
 - Grammaire régulière gauche : analyse ascendante
↔ Du mot vers l'axiome

Grammaires régulières droite et gauche (2/2)

Exemple : $G_1 = \{T, N_1, S_1, R_1\}$ et $G_2 = \{T, N_2, S_2, R_2\}$ avec $T = \{a, b\}$ engendrent le même langage $L = \{a^n b^m / n > 0 \wedge m > 0\}$ où :

$$N_1 = \{S_1, U_1\}$$

$$R_1 = \begin{array}{l} S_1 \rightarrow aS_1 \mid aU_1 \\ U_1 \rightarrow bU_1 \mid b \end{array}$$

$$N_2 = \{S_2, U_2\}$$

$$R_2 = \begin{array}{l} S_2 \rightarrow S_2 b \mid U_2 b \\ U_2 \rightarrow U_2 a \mid a \end{array}$$

L'analyse du mot $aaabb$ avec G_1 :

$$S_1 \Rightarrow aS_1 \Rightarrow aaS_1 \Rightarrow aaaU_1 \Rightarrow aaabU_1 \Rightarrow aaabb$$

L'analyse du mot $aaabb$ avec G_2 :

$$S_2 \Rightarrow S_2 b \Rightarrow U_2 bb \Rightarrow U_2 abb \Rightarrow U_2 aabb \Rightarrow aaabb$$

Langage régulier et automate régulier

Théorème

Tout langage accepté par un automate fini est régulier.

Théorème

Tout langage régulier est accepté par un automate fini.

Remarque

À démontrer en TD !

Grammaire contextuelle

- Certains langages ne sont pas engendrés par une grammaire hors-contexte
- Exemple : $L = \{w \mid w = a^n b^n c^n, n \geq 0\}$
 $\hookrightarrow$ Possible de définir une grammaire sous contexte pour ce langage

Définition : grammaire sous contexte

Une grammaire sous contexte (ou contextuelle) est définie par $G = \{T, N, R, S\}$ où les règles de R sont de la forme suivante :

$$uAv \rightarrow uwv \text{ où } A \in N, u, v \in (T \cup N)^*, w \in (T \cup N)^+$$

Exemple de grammaire sous contexte

Soit la grammaire sous contexte :

$$\begin{array}{lll} S & \rightarrow & aAb \\ aA & \rightarrow & aaAb \\ A & \rightarrow & \epsilon \end{array}$$

Le langage est $L(G) = \{a^n b^n, n \geq 1\}$.

Exemple de dérivation avec le mot *aaabbb* :

$$S \Rightarrow aAb \Rightarrow aaAbb \Rightarrow aaaAbbb \Rightarrow aaabbb$$

Arbre d'analyse / syntaxique

Définition : arbre d'analyse

Un arbre d'analyse (ou syntaxique) pour $G = \{T, N, S, R\}$ est un arbre dont chaque nœud est étiqueté par un élément de $(T \cup N \cup \epsilon)$ et qui satisfait les conditions suivantes :

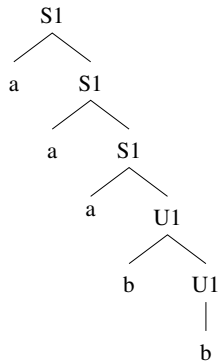
- *La racine est étiquetée par l'axiome S*
- *Chaque nœud inférieur est étiqueté par un non-terminal ($\in N$) et chaque feuille par un symbole terminal ($\in T$) ou par ϵ*
- *Pour tout nœud intérieur, si son étiquette est $A \in N$ et si ses fils sont les nœuds ayant pour étiquettes $X_1, \dots, X_k$ alors : $A \rightarrow X_1 X_2 \dots X_k \in R$*
- *Si un nœud est étiqueté par ϵ , il est le seul fils de son père*

Exemple

Pour la dérivation :

$$S_1 \Rightarrow aS_1 \Rightarrow aaS_1 \Rightarrow aaaU_1 \Rightarrow aaabU_1 \Rightarrow aaabb$$

L'arbre syntaxique correspondant est :



Mot généré par un arbre d'analyse

Définition : mot généré par un arbre d'analyse

Le mot généré par un arbre d'analyse est celui obtenu par la concaténation des feuilles de l'arbre de gauche à droite.

Théorème

Étant donnée une grammaire hors-contexte G , un mot w est généré par G ($S \Rightarrow^ w$) si et seulement s'il existe un arbre d'analyse de la grammaire G qui génère w .*

Exemple avec les expressions arithmétiques (1/2)

Soit la grammaire $G = \{T, N, R, E\}$ avec R :

$$E \rightarrow \text{cst} \mid id \mid E + E \mid E * E \mid (E)$$

Première dérivation possible du mot $2 * x + y$:

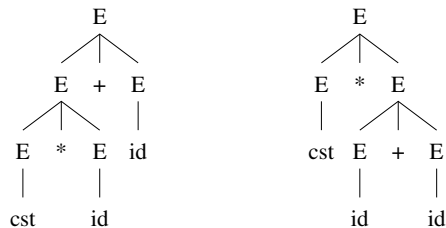
$$E \Rightarrow E + E \Rightarrow E * E + E \Rightarrow E * id + E \Rightarrow \text{cst} * id + E \Rightarrow \text{cst} * id + id$$

Deuxième dérivation possible :

$$E \Rightarrow E * E \Rightarrow E * E + E \Rightarrow \text{cst} * E + E \Rightarrow \text{cst} * id + E \Rightarrow \text{cst} * id + id$$

Exemple avec les expressions arithmétiques (2/2)

Les deux arbres syntaxiques correspondant aux dérivations :



- Deux arbres syntaxiques pour le même mot : ambiguïté!
↪ Mathématiquement, seul celui de gauche est "correct"
- Cela nécessite l'utilisation de parenthèses ou la définition de priorité

Grammaires ambiguës

Définition : grammaire ambiguë

Une grammaire est ambiguë si plus d'un arbre syntaxique est généré pour un même mot.

- Il n'existe pas d'algorithme permettant de vérifier si une grammaire est ambiguë
 $\hookrightarrow$ C'est un problème indécidable
- Cependant, il existe des familles de grammaires démontrées non ambiguës

Les expressions sans ambiguïté

Soit la grammaire $G = \{T, N, R, E\}$ avec R :

$$E \rightarrow E + E \mid E * E$$

$$E \rightarrow (E)$$

$$E \rightarrow \text{cst} \mid \text{id}$$

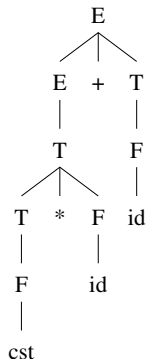
devient

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{cst} \mid \text{id} \mid (E)$$

Dérivation du mot $2 * x + y$:



Autre exemple : les conditionnelles (1/3)

Voici la grammaire ambiguë qui reconnaît une conditionnelle :

$$\begin{aligned} inst &\rightarrow Si\ expr\ Alors\ inst \\ &\rightarrow Si\ expr\ Alors\ inst\ Sinon\ inst \\ &\rightarrow \text{autres instructions} \end{aligned}$$

Avec $T = \{Si, Alors, Sinon, \dots\}$ et $N = \{inst, expr, \dots\}$

Comment reconnaître le mot suivant ?

Si cond1 Alors Si cond2 Alors inst1 Sinon inst2

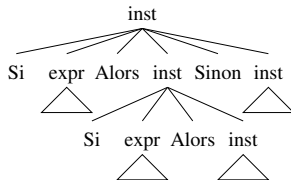
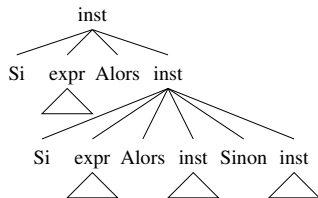
Autre exemple : les conditionnelles (2/3)

Deux dérivations possibles (celle de gauche est préférée habituellement) :

Si cond1 **Alors**
 Si cond2 **Alors**
 inst1
 Sinon
 inst2

Si cond1 **Alors**
 Si cond2 **Alors**
 inst1
 Sinon
 inst2

Les arbres syntaxiques correspondants :



Autre exemple : les conditionnelles (3/3)

Voici une grammaire qui lève l'ambiguïté :

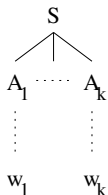
$$\begin{array}{ll}
 inst & \rightarrow \quad inst_close \\
 & \quad | \quad inst_non_close \\
 inst_close & \rightarrow \quad Si\ expr\ Alors\ inst_close\ Sinon\ inst_close \\
 & \quad | \quad autre \\
 inst_non_close & \rightarrow \quad Si\ expr\ Alors\ inst \\
 & \quad | \quad Si\ expr\ Alors\ inst_close\ Sinon\ inst_non_close
 \end{array}$$

- Permet de faire correspondre le *sinon* avec le *alors* précédent
 $\hookrightarrow$ Comme la majorité des langages de programmation actuels
- Instruction close :
 - Instruction si/alors/sinon sans instruction non close
 - Instruction non conditionnelle

Dérivation gauche et droite

- À partir d'une grammaire, il est possible de construire automatiquement un analyseur syntaxique
- Lorsque plusieurs dérivations sont possibles :
 - Analyseur gauche : prend la dérivation la plus à gauche possible
 - Analyseur droit : prend la dérivation la plus à droite possible

Exemple



- Soit la dérivation :
 $S \Rightarrow A_1 \dots A_k \Rightarrow^* w_1 \dots w_k$
- À l'étape $A_1 \dots A_k$:
 - $\hookrightarrow A_1$: dérivation la plus à gauche
 - $\hookrightarrow A_k$: dérivation la plus à droite

Comparaison entre dérivation gauche et droite

Soit la grammaire des expressions :

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

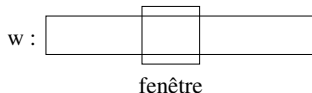
$$F \rightarrow \text{cst} \mid \text{id} \mid (E)$$

Mot à analyser : $\text{id} + \text{id} * \text{id}$

Dérivation gauche	Dérivation droite
E	E
E + T	E + T
T + T	E + T * F
F + T	E + T * id
id + T	E + F * id
id + T * F	E + id * id
id + F * F	T + id * id
id + id * F	F + id * id
id + id * id	id + id * id

Analyse syntaxique descendante

- Vue comme une tentative pour déterminer une dérivation gauche d'un mot
- Construction de l'arbre :
 - On part de la racine
 - On construit les nœuds en préordre
- Idée : on avance le plus possible dans le mot



Problème du rebroussement

Soit la grammaire suivante :

$$S \rightarrow cAd$$

$$A \rightarrow ab \mid a$$

- Que faire lorsqu'un a est rencontré ?
- Si la "mauvaise règle" est utilisée, un **rebroussement** est nécessaire
 $\hookrightarrow$ L'analyse descendante réursive
- Généralement, peu fréquents car peu efficaces
 - Possible de l'éviter avec la plupart des langages
 - Réécriture de la grammaire si possible
- Un analyseur sans rebroussement est appelé **analyseur prédictif**

Problème de la récursivité (gauche)

Définition : grammaire récursive gauche

Une grammaire est récursive gauche si elle contient $A \in N$ tel que :

$$A \Rightarrow^+ A\alpha \text{ avec } \alpha \in (T \cup N)^*$$

- La construction d'un analyseur récursif est impossible
 - La procédure de reconnaissance de A s'appelle indéfiniment
 - La fenêtre ne sera jamais modifiée
- ↪ Exécution infinie

⇒ Il faut supprimer la récursivité gauche !

Élimination de la récursivité gauche (immédiate)

- Soit G une grammaire récursive gauche dont les règles sont de la forme :

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n$$

$$A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$$

- Pour éliminer la récursivité, nous réécrivons les règles comme suit :

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$$

Exemple (1/2)

Soit la grammaire des expressions :

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow \text{cst} \mid \text{id} \mid (E) \end{aligned}$$

Nous devons supprimer la récursivité pour E , puis pour T .

- Si $A = E$, $\alpha_1 = +T$, $\beta_1 = T$
 - $A \rightarrow \beta_1 A'$ devient $E \rightarrow TE'$
 - $A' \rightarrow \alpha_1 A' | \epsilon$ devient $E' \rightarrow +TE' | \epsilon$
- Si $A = T$, $\alpha_1 = *F$, $\beta_1 = F$
 - $A \rightarrow \beta_1 A'$ devient $T \rightarrow FT'$
 - $A' \rightarrow \alpha_1 A' | \epsilon$ devient $T' \rightarrow *FT' | \epsilon$

Exemple (2/2)

La grammaire des expressions :

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow \text{cst} \mid \text{id} \mid (E) \end{aligned}$$

Devient :

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow \text{cst} \mid \text{id} \mid (E) \end{aligned}$$

Problème de la récursivité profonde

- L'absence de récursivité immédiate n'est pas suffisante
- Exemple :

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- Possible d'avoir les dérivations suivantes : $S \Rightarrow Aa \Rightarrow Sda$
 $\hookrightarrow$ La grammaire est récursive
- Solution : utiliser un algorithme pour l'éliminer

Grammaire sans production vide

- $v \rightarrow \epsilon$ est appelée une **production vide**

Définition : grammaire sans production vide

Une grammaire est sans production vide si :

- *Elle n'a pas de production vide*
- *Elle a une production vide $S \rightarrow \epsilon$ où S est l'axiome et n'apparaît pas dans les parties droites des productions*

Remarque

Il est possible de transformer une grammaire avec productions vides en une grammaire sans production vide.

Algorithme d'élimination de la récursivité gauche

Algorithme pour une grammaire sans cycle et sans production vide

Déterminer un ordre pour les non terminaux notés $A_1 \dots A_n$

Pour i allant de 1 à n **Faire**

Pour j allant de 1 à $i - 1$ **Faire**

 Remplacer productions de la forme $A_i \rightarrow A_j \alpha$ par $A_i \rightarrow \beta_1 \alpha \mid \dots \mid \beta_k \alpha$ où $A_j \rightarrow \beta_1 \dots \beta_k$

Fin Pour

 Éliminer la récursivité gauche immédiate des A_i productions

Fin Pour

Éliminer les symboles grammaticaux qui ne servent plus

Remarque

Dans certains cas, il est possible d'utiliser cet algorithme pour des grammaires ayant des productions vides.

Exemple d'élimination de la récursivité gauche

Soit :

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- Choix d'un ordre ; exemple : S, A donc $A_1 = S$ et $A_2 = A$
- Pour $i = 1$:
 - On ne rentre pas dans la deuxième boucle
 - Pas de récursivité gauche immédiate pour S
- Pour $i = 2$:
 - Remplacement de $A \rightarrow Sd$ par des $A \rightarrow Aad \mid bd$
 - La grammaire devient :

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Aad \mid bd \mid \epsilon \end{aligned}$$

- Suppression de la récursivité gauche immédiate de A

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \epsilon \end{aligned}$$

Factorisation gauche : présentation de la problématique

Soit :

$$\begin{aligned} inst &\rightarrow \text{Si } expr \text{ Alors } inst \\ &\rightarrow \text{Si } expr \text{ Alors } inst \text{ Sinon } inst \\ &\rightarrow \text{autres instructions} \end{aligned}$$

Avec $T = \{Si, Alors, Sinon, \dots\}$ et $N = \{inst, expr, \dots\}$

- Quelle règle de production choisir lorsqu'un 'Si' est rencontré ?
- Choix une fois le 'Sinon' rencontré ou pas
- Solution : factoriser la grammaire

Factorisation gauche

Si une grammaire contient des règles de la forme :

$$\begin{aligned} A &\rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_n \\ &\rightarrow \gamma_1 \mid \dots \mid \gamma_m \end{aligned}$$

Nous pouvons factoriser à gauche comme suit :

$$\begin{aligned} A &\rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_m \\ A' &\rightarrow \beta_1 \mid \dots \mid \beta_n \end{aligned}$$

Exemple de factorisation à gauche

Soit :

$$\begin{aligned} S &\rightarrow iEtS \mid iEtSeS \mid a \\ E &\rightarrow b \end{aligned}$$

- Correspond à la grammaire des conditionnelles :
 - i correspond à 'Si', t à 'Alors' et e à 'Sinon'
 - E est une expression, S une instruction
- Après la factorisation :

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

- Sur un 'Si', on peut maintenant développer $iEtSS'$
- Une fois $iEtSS'$ reconnu : on développe eS ou ϵ

Analyseur prédictif

- Supposons une grammaire :
 - Sans récursivité immédiate/profonde à gauche
 - Factorisée à gauche
- On souhaite analyser cette grammaire par descente récursive sans rebroussement
↪ Construction d'un analyseur prédictif
- Problématique de l'analyseur :
 - Soit un symbole terminal a en entrée
 - Soit le symbole A à développer↪ Quelle production parmi $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$ doit-on développer ?
- Il faut déterminer :
 - ① Quel α peut commencer par un a ?
 - ② Si un α accepte le mot vide, il faut pousser la recherche sur les symboles que l'on peut rencontrer après un A

Fonctions *Premier* et *Suivant*

Définition : fonction *Premier*

Soit $G = (T, N, R, S)$. Si $\alpha \in (T \cup N)^*$, $\text{Premier}(\alpha)$ est l'ensemble des symboles terminaux qui commencent les chaînes qui se dérivent de α :

$$\text{Premier}(\alpha) = \{x \in T \mid \exists \beta \in (T \cup N)^*, \alpha \Rightarrow^* x\beta\} \cup \{\epsilon \text{ si } \alpha \Rightarrow^* \epsilon\}$$

Définition : fonction *Suivant*

Soit $G = (T, N, R, S)$. Si $A \in N$, $\text{Suivant}(A)$ est l'ensemble des symboles terminaux qui peuvent apparaître directement à droite de A dans une protophrase :

$$\text{Suivant}(A) = \{x \in T \mid \exists \alpha, \beta \in (T \cup N)^*, S \Rightarrow^* \alpha A x \beta\} \cup \{\epsilon \text{ si } S \Rightarrow^* \alpha A\}$$

Algorithme de calcul pour *Premier*

Initialisation :

- $\forall X \in T, Premier(X) = \{X\}$
- $\forall X \in N, Premier(X) = \emptyset$

Pour tout $X \in N$ et $X \rightarrow Y_1 Y_2 \dots Y_n$, appliquer ces règles jusqu'à ce qu'aucun terminal, ni ϵ ne puisse être ajouté aux ensembles *Premier* :

- Si $\exists i \in [1, n] / a \neq \epsilon \in Premier(Y_i)$ et $\forall j \in [1, i - 1] \epsilon \in Premier(Y_j)$ alors $a \in Premier(X)$;
- Si $\forall i \in [1, n], \epsilon \in Premier(Y_i)$ alors $\epsilon \in Premier(X)$

Remarques

- Si $X \rightarrow \epsilon$, alors $\epsilon \in Premier(X)$
- Si $X \rightarrow aY$ avec $a \in T$, alors $a \in Premier(X)$

Algorithme de calcul pour *Suivant*

Initialisation :

- $\forall A \in N, \text{Suivant}(A) = \emptyset$

On applique les règles suivantes :

- $\epsilon \in \text{Suivant}(S)$ si S est l'axiome
- Si $A \rightarrow \alpha B \beta$ où $A, B \in N$ et $\alpha, \beta \in (T \cup N)^*$:
 $\text{Suivant}(B) = \text{Suivant}(B) \cup \text{Premier}(\beta) \setminus \epsilon$
- Si $A \rightarrow \alpha B$ ou $A \rightarrow \alpha B \beta$ avec $\epsilon \in \text{Premier}(\beta)$:
 $\text{Suivant}(B) = \text{Suivant}(B) \cup \text{Suivant}(A)$

Exemples de calcul de *Premier* et de *Suivant*

Soit :

$$\begin{array}{lcl} S & \rightarrow & X Y \mid d \\ Y & \rightarrow & c \mid \epsilon \\ X & \rightarrow & Y \mid a \end{array}$$

- $Premier(S) = \{a, c, d, \epsilon\}$
- $Premier(X) = \{a, c, \epsilon\}$
- $Premier(Y) = \{c, \epsilon\}$
- $Suivant(S) = \{\epsilon\}$
- $Suivant(X) = \{c, \epsilon\}$
- $Suivant(Y) = \{c, \epsilon\}$

Grammaires LL(1) : caractérisation

- Soit une grammaire $G = (T, N, R, S)$ hors-contexte, possédant des règles de la forme $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$
- Pour construire un analyseur prédictif descendant sans rebroussement, il est nécessaire de pouvoir choisir la règle de production ($A \rightarrow \alpha_i, 1 \leq i \leq n$) en fonction de la fenêtre (le prochain lexème à analyser)
 - ↪ Une seule règle possible
- Plusieurs conditions :
 - Ensembles $Premier(\alpha_i)$ disjoints 2 à 2
 - ↪ Le symbole actuel nous permet de choisir une seule règle
 - Si $\epsilon \in Premier(A)$, la fenêtre est élément de $Suivant(A)$; chaque $Premier(\alpha_i)$ contenant ϵ est disjoint de $Suivant(A)$
 - ↪ Pas de confusion entre la réduction de A et de α_i

Grammaires LL(1)

Définition : grammaire LL(1)

$G = (T, N, R, S)$ est une grammaire LL(1) si et seulement si :

$$\forall A \in N / A \rightarrow \alpha \in R \wedge A \rightarrow \beta \in R, (A \rightarrow \alpha \neq A \rightarrow \beta) \Rightarrow \\ Premier(\alpha) \cap Premier(\beta) = \emptyset \text{ et } \epsilon \in Suivant(\beta) \Rightarrow Premier(\alpha) \cap Suivant(A) = \emptyset$$

Remarques

- Il y a au plus une seule règle de A qui dérive sur ϵ
- Si $\alpha \Rightarrow^* \epsilon$, β ne dérive pas sur une chaîne commençant par $Suivant(A)$

Analyseur syntaxique récursif descendant (1/3)

Soit $G = (T, N, R, S)$, une grammaire LL(1)

- Augmentation de la grammaire par un $S' \in N$ qui est le nouvel axiome :

$$G' = (T, N \cup \{S'\}, R \cup \{S' \rightarrow S\}, S')$$

- La procédure pour S' est la suivante :

Procédure $S'()$

$S()$

Si symbole $\neq \vdash$ **Alors**

Erreur

Analyseur syntaxique récursif descendant (2/3)

- Pour chaque règle $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$ avec $\epsilon \notin \text{Premier}(A)$, on construit une procédure
- Si $\epsilon \in \text{Premier}(A)$, il faut réduire A à ϵ pour tout $a \in T$ tel que $\forall 1 \leq i \leq n, a \notin \text{Premier}(\alpha_i)$

Procédure $A()$

Si symbole $\in \text{Premier}(\alpha_1)$ **Alors**

$\alpha_1()$

Sinon Si symbole $\in \text{Premier}(\alpha_2)$ **Alors**

$\alpha_2()$

Sinon

Si $\epsilon \notin \text{Premier}(A)$ **Alors**

Erreur

Sinon

/ A est réduit à ϵ */*

Analyseur syntaxique récursif descendant (3/3)

- Pour chaque α_i , on construit une procédure
- Si $\alpha_i \rightarrow a\beta$ et $a \in T$, déplacement de la fenêtre avant de reconnaître β

Procédure $\alpha_i()$

/ symbole = a, avec $a \in Premier(A)$ */*

avance

$\beta()$

- Si $\alpha_i \rightarrow AB$ avec $A \in N$, la fenêtre n'est pas modifiée

Procédure $\alpha_i()$

A()

B()

- Si $\alpha_i \rightarrow \epsilon$ alors la procédure est vide

Exemple (1/3)

Soit la grammaire :

$$\begin{aligned} S &\rightarrow (A \\ A &\rightarrow B) \\ &\rightarrow) \\ B &\rightarrow \text{entier } C \\ &\rightarrow S C \\ C &\rightarrow , B \\ &\rightarrow \epsilon \end{aligned}$$

$$S' \rightarrow S$$

Procédure $S'()$

$S()$

Si symbole $\neq \mid$ **Alors**

Erreur

$$S \rightarrow (A$$

Procédure $S()$

Si symbole = '(' **Alors**
avance; $A()$

Sinon

erreur

Exemple (2/3)

$A \rightarrow B) \mid)$

Procédure A()

Si symbole $\in Premier(B) = \{ entier, '(' \}$ **Alors**

 B()

Si symbole = ')' **Alors**

 avance

Sinon

 erreur

Sinon

Si symbole = ')' **Alors**

 avance

Sinon

 erreur

Exemple (3/3)

$B \rightarrow \text{entier } C \mid SC$

Procédure B()

Si symbole = entier **Alors**

 avance

 C()

Sinon Si symbole = '(' **Alors**

 S()

 C()

Sinon

 erreur

Avec $\text{Premier}(S) = \{ '(' \}$

$C \rightarrow , B \mid \epsilon$

Procédure C()

Si symbole = ',' **Alors**

 avance

 B()

Sinon

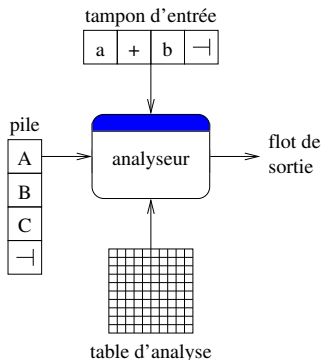
 /* Rien */

Optimisations

- Remplacer les procédures appelées une fois par leur corps
- Éliminer des textes de programme communs
- Éliminer les appels récursifs terminaux à l'aide de boucles TantQue

Analyseur LL

- Une autre solution pour analyser une grammaire LL(1) : utiliser un analyseur LL



- Tampon d'entrée : chaîne à analyser terminée par $\perp$
- Pile : contient des symboles grammaticaux
 $\hookrightarrow$ Initialement : axiome et $\perp$
- Table d'analyse : tableau à deux dimensions $M[A, a]$ avec
 $A \in N, a \in T \cup \{\perp\}$
 $\hookrightarrow$ En fonction du sommet de la pile et du prochain lexème, action déterminée par la table

Exécution d'un analyseur LL

En fonction du symbole X de la pile et de a le symbole d'entrée :

- $X = a = \perp$: analyse réussie
 $\hookrightarrow$ Action **ACC** (ACCepter)
- $X = a \neq \perp$: X dépilé, pointeur d'entrée avancé
 $\hookrightarrow$ Action **AVCTS** (AVancer dans la Cible et dans le Texte Source)
- Si $X \in N$, consultation de la table $M[X, a]$, deux possibilités :
 - Erreur
 - Production $X \rightarrow UVW$, X remplacé par W , V et U (U au sommet de la pile)
 $\hookrightarrow$ Si production vide : action **AVC** (AVancer dans la Cible)

La construction de la table a besoin des ensembles *Premier* et *Suivant*

Construction de la table d'analyse

Nous distinguons les 4 règles suivantes :

- ① $M[\neg, \neg] = \text{ACC}$
 $\hookrightarrow$ Le mot a été reconnu
- ② $\forall a \in T, M[a, a] = \text{AVCTS}$
 $\hookrightarrow$ Une unité lexicale a été reconnue, il passe à la suite
- ③ $\forall A \in N$, s'il existe une règle $A \rightarrow \alpha$ (avec $\alpha \neq \epsilon$), $M[A, a] = A \rightarrow \alpha$,
 $\forall a \in \text{Premier}(\alpha) \cup \text{Suivant}(A)$ si $\alpha \Rightarrow^* \epsilon$
 $\hookrightarrow$ Une production est développable
- ④ $\forall A \in N$, s'il existe une règle $A \rightarrow \epsilon$, $M[A, a] = \text{AVC}$, $\forall a \in \text{Suivant}(A) \setminus \{\epsilon\} \cup \{\neg\}$ si $\epsilon \in \text{Suivant}(A)$
 $\hookrightarrow A$ doit être réduit par ϵ si on trouve un suivant de A
 $\hookrightarrow$ Si ϵ appartient à $\text{Suivant}(A)$, alors on le réduit également par ϵ si on trouve $\neg$

Construction de la table d'analyse : exemple

$S \rightarrow (A$
 $A \rightarrow B)$
 $\rightarrow)$
 $B \rightarrow \text{entier } C$
 $\rightarrow S C$
 $C \rightarrow , B$
 $\rightarrow \epsilon$

- *Premier* :

- $\text{Premier}(S) = \{ '(' \}$
- $\text{Premier}(A) = \{ \text{entier}, '(', ')' \}$
- $\text{Premier}(B) = \{ \text{entier}, '(' \}$
- $\text{Premier}(C) = \{ ', ', \epsilon \}$

- *Suivant* :

- $\text{Suivant}(S) = \{ ', ', ')', \epsilon \}$
- $\text{Suivant}(A) = \{ ', ', ')', \epsilon \}$
- $\text{Suivant}(B) = \{ ')', \epsilon \}$
- $\text{Suivant}(C) = \{ ')', \epsilon \}$

Construction de la table

	'('	')'	<i>entier</i>	','	⊢
<i>S</i>	$S \rightarrow (A$				
<i>A</i>	$A \rightarrow B)$	$A \rightarrow)$	$A \rightarrow L)$		
<i>B</i>	$B \rightarrow SC$		$B \rightarrow aC$		
<i>C</i>		AVC		$C \rightarrow , B$	
'('	AVCTS				
')'		AVCTS			
<i>entier</i>			AVCTS		
','				AVCTS	
⊢					ACC

Exemple d'exécution

- Analyse de : (1,2)

Cible	Texte source	Action
$S \vdash$	$(1, 2) \vdash$	$S \rightarrow (A$
$(A \vdash$	$(1, 2) \vdash$	AVCTS
$A \vdash$	$1, 2) \vdash$	$A \rightarrow B)$
$B) \vdash$	$1, 2) \vdash$	$B \rightarrow \text{entier } C$
$\text{entier } C) \vdash$	$1, 2) \vdash$	AVCTS
$C) \vdash$	$, 2) \vdash$	$C \rightarrow , B$
$, B) \vdash$	$, 2) \vdash$	AVCTS
$B) \vdash$	$2) \vdash$	$B \rightarrow \text{entier } C$
$\text{entier } C) \vdash$	$2) \vdash$	AVCTS
$C) \vdash$	$) \vdash$	AVC
$) \vdash$	$) \vdash$	AVCTS
$\vdash$	$\vdash$	ACC